

TỔNG QUAN VỀ DEVOPS



Mục tiêu buổi học

- 1) Vì sao cách làm phần mềm truyền thống gặp nhiều vấn đề?
- 2) DevOps là gì và ra đời để giải quyết vấn đề gì?
- 3) DevOps không là gì?



Nội dung

1

Những vấn đề của phát triển phần mềm truyền thống

2

Dev và Ops – Tư duy DevOps

3

DevOps với Agile và Lean

4

DevOps với security - DevSecOps

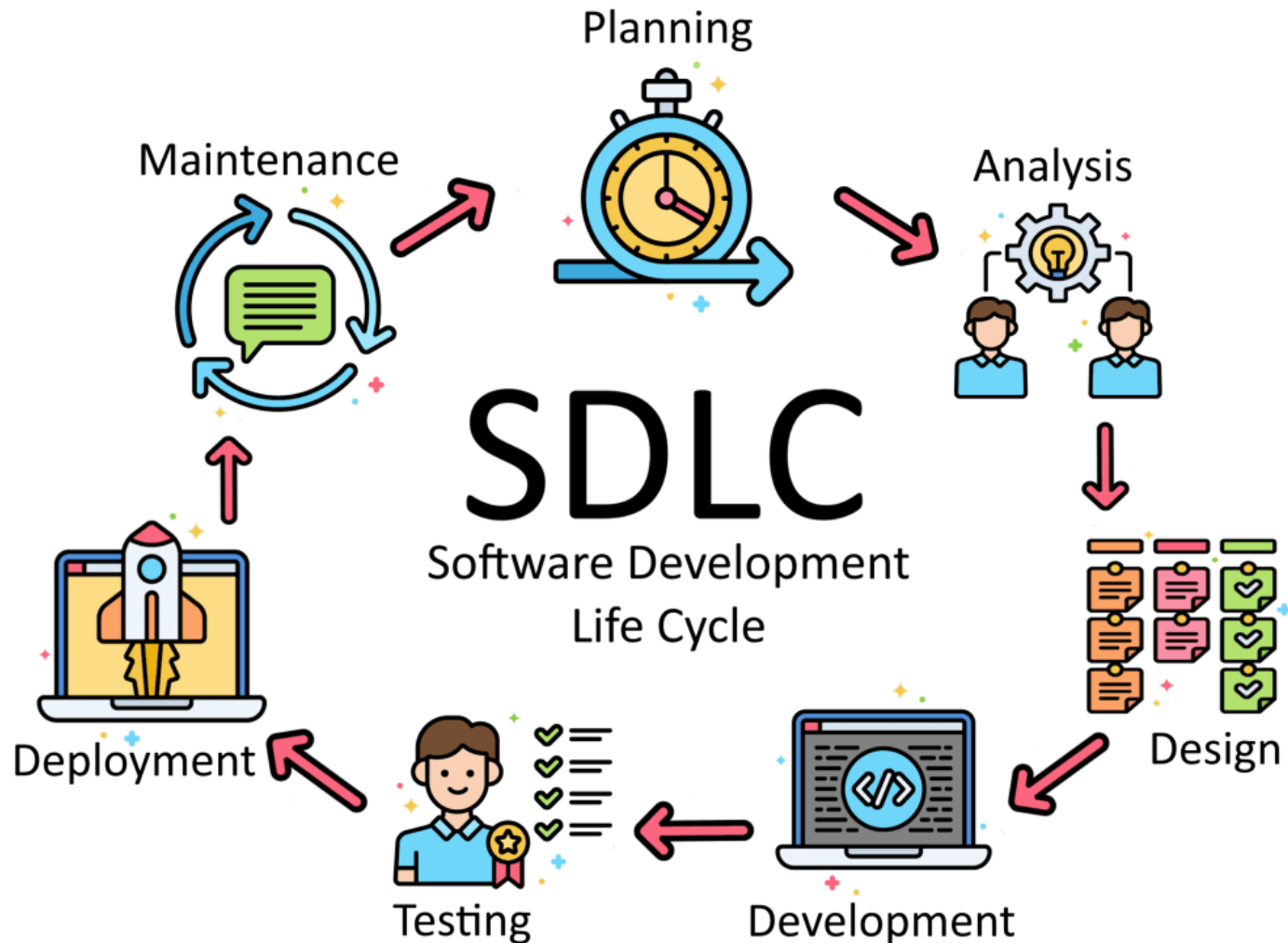
Dẫn nhập

Vì sao phần mềm chạy tốt trên máy Dev nhưng lại lỗi trên Production?

Có vấn đề trong hệ thống phát triển phần mềm

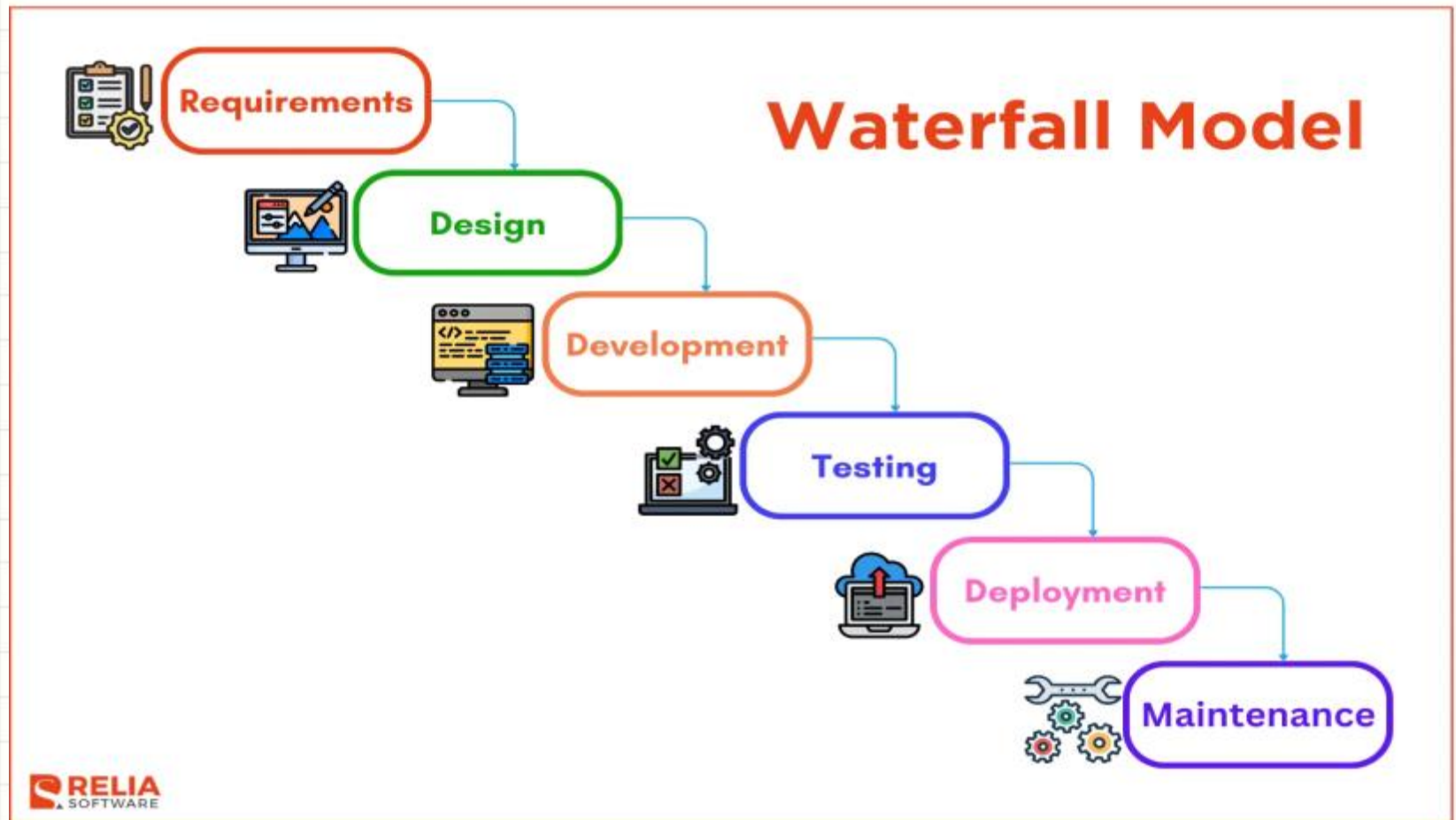
Những vấn đề của phát triển phần mềm truyền thống

Vòng đời phát triển phần mềm (SDLC)



Những vấn đề của phát triển phần mềm truyền thống

Mô hình phát triển phần mềm Waterfall



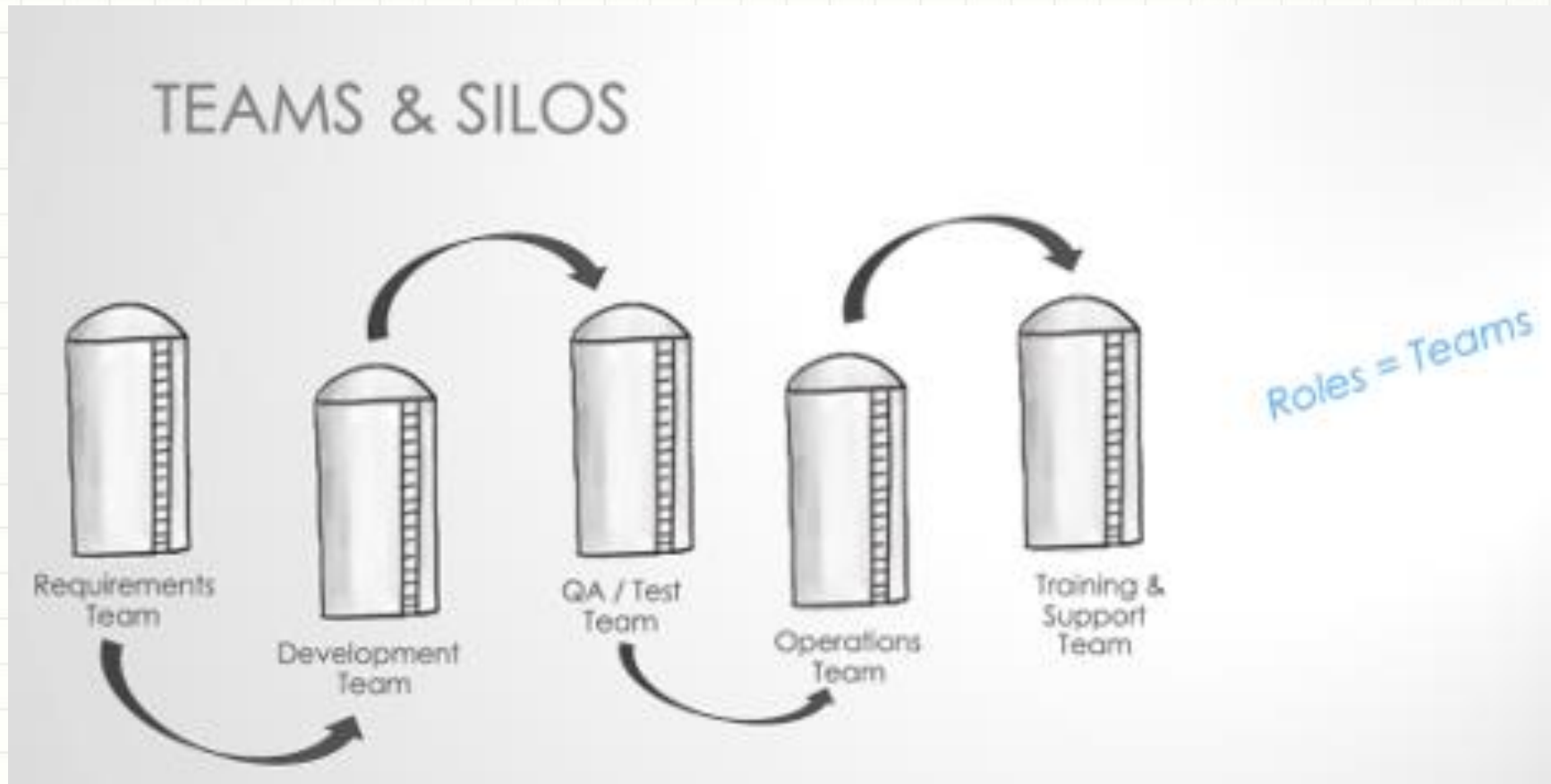
Những vấn đề của phát triển phần mềm truyền thống

Tổ chức kiểu silo



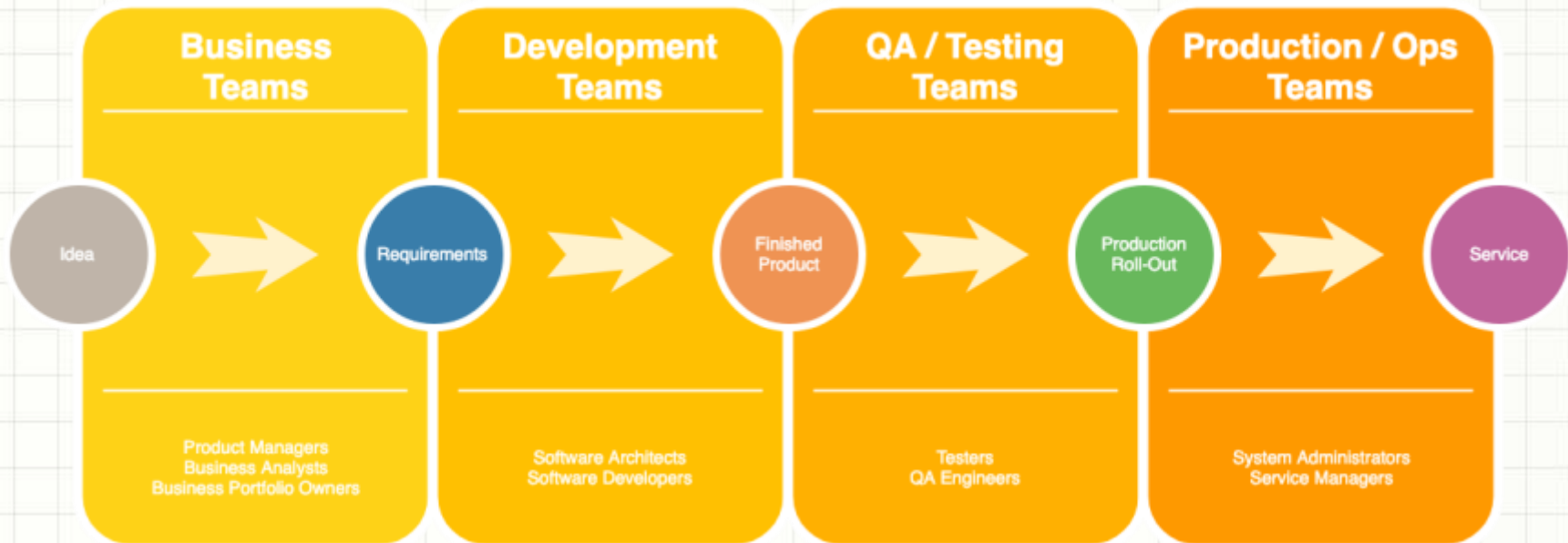
Những vấn đề của phát triển phần mềm truyền thống

Tổ chức kiểu silo



Những vấn đề của phát triển phần mềm truyền thống

Tổ chức kiểu silo



☞ Silo = phân mảnh + cô lập + thiếu phối hợp

☞ Mỗi team tối ưu phần việc của mình, nhưng **không ai tối ưu toàn hệ thống**

Những vấn đề của phát triển phần mềm truyền thống

Dấu hiệu cho thấy một tổ chức kiểu silo:

- Tài liệu không chia sẻ, người khác muốn biết phải “xin”
- Khi sự cố xảy ra, mọi người hỏi: “team nào chịu trách nhiệm?” trước khi hỏi “vì sao xảy ra?”
- Mỗi team có KPI riêng, đôi khi **mâu thuẫn trực tiếp** với nhau
- Deploy cần nhiều người “ký duyệt” vì thiếu niềm tin liên nhóm

Những vấn đề của phát triển phần mềm truyền thống

Hệ quả: tạo ra nút thắt giao tiếp

➤ **Bàn giao & chậm trễ:**

Các nhóm làm việc độc lập, chuyển giao qua nhiều giai đoạn → phát sinh chậm trễ và phải làm lại.

➤ **Phản hồi hạn chế:**

Dev không nhận được phản hồi kịp thời → lỗi tồn tại đến các giai đoạn sau.

➤ **Đổ lỗi lẫn nhau:**

Khi lỗi xảy ra trên production, không rõ trách nhiệm thuộc Dev hay Ops → dễ xảy ra “đổ lỗi”.

Những vấn đề của phát triển phần mềm truyền thống

Hạn chế của tiếp cận silo:

➤ Ra thị trường chậm:

Giao tiếp vòng vo, sửa đi sửa lại làm chậm quá trình release.

➤ Chất lượng phần mềm giảm:

Ít test, thiếu trao đổi → nhiều bug lọt lên production.

➤ Xung đột giữa các nhóm:

Tư duy “làm xong rồi ném qua tường” tạo quan hệ đối đầu giữa Dev và Ops.

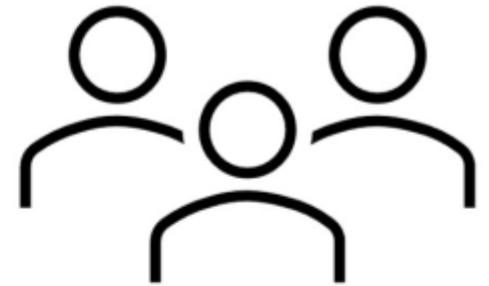
👉 Cần một cách tiếp cận tốt hơn

Dev vs Ops – Tư duy DevOps

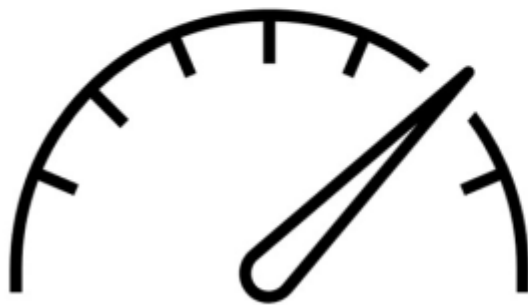
Dev vs Ops:



Development Team



Operations Team



Objective: Speed!



Objective: Stability!

Dev vs Ops – Tư duy DevOps

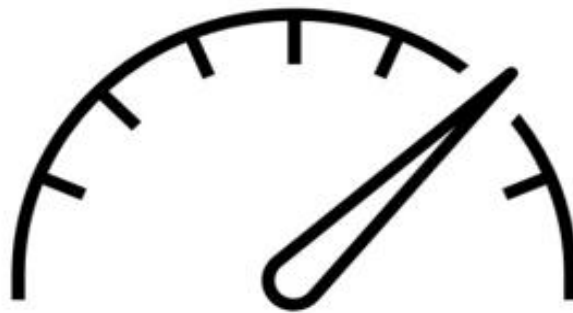
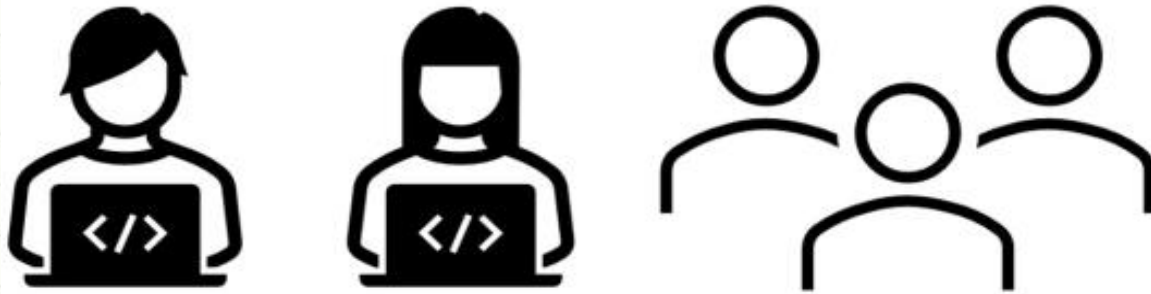
Tư duy DevOps:

- Không xóa Dev
- Không xóa Ops
- Kết nối Dev với Ops vì mục tiêu chung

👉 DevOps không phải là “team mới” mà là cách làm mới

Dev vs Ops – Tư duy DevOps

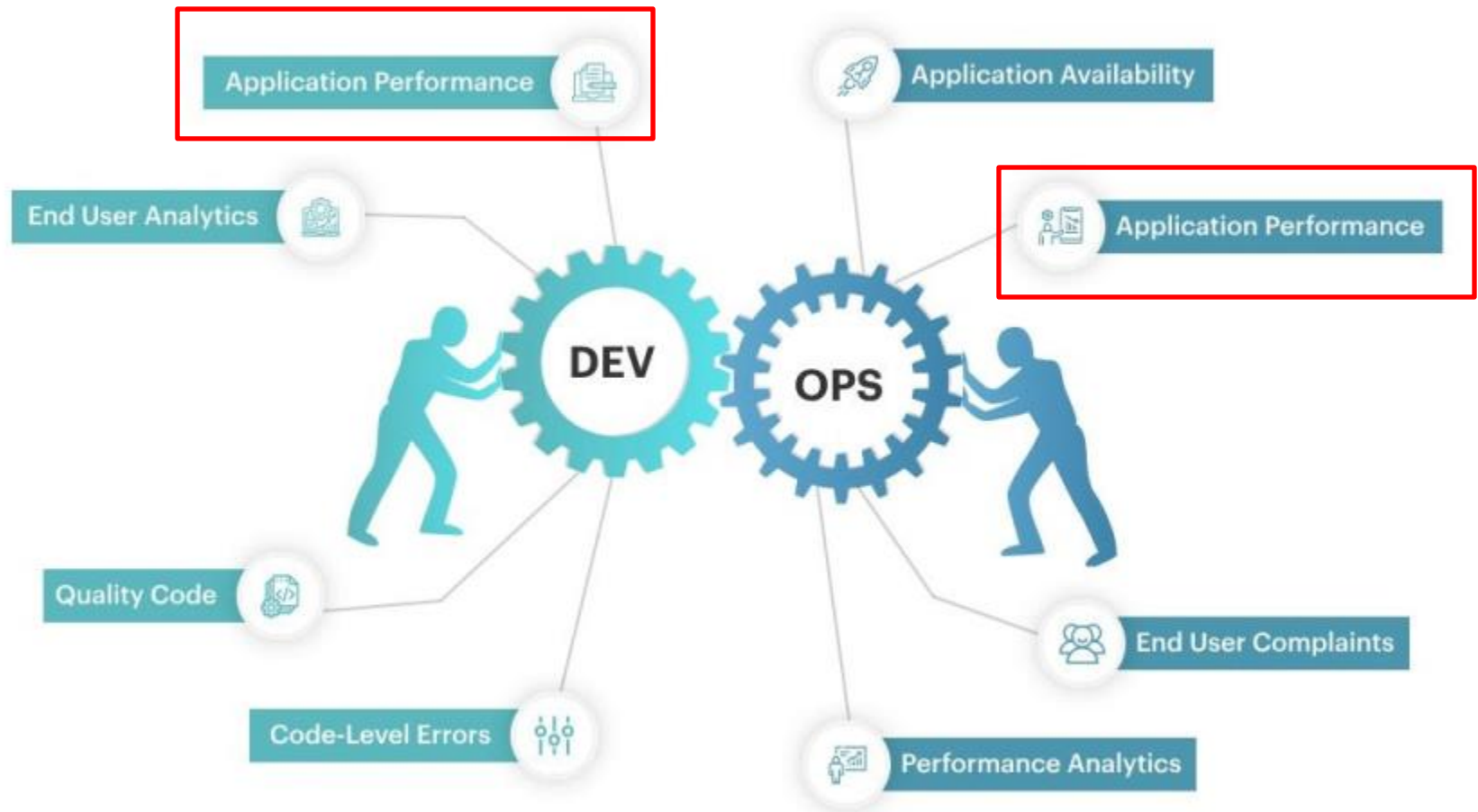
Tư duy DevOps:



Objective: Speed and Stability!

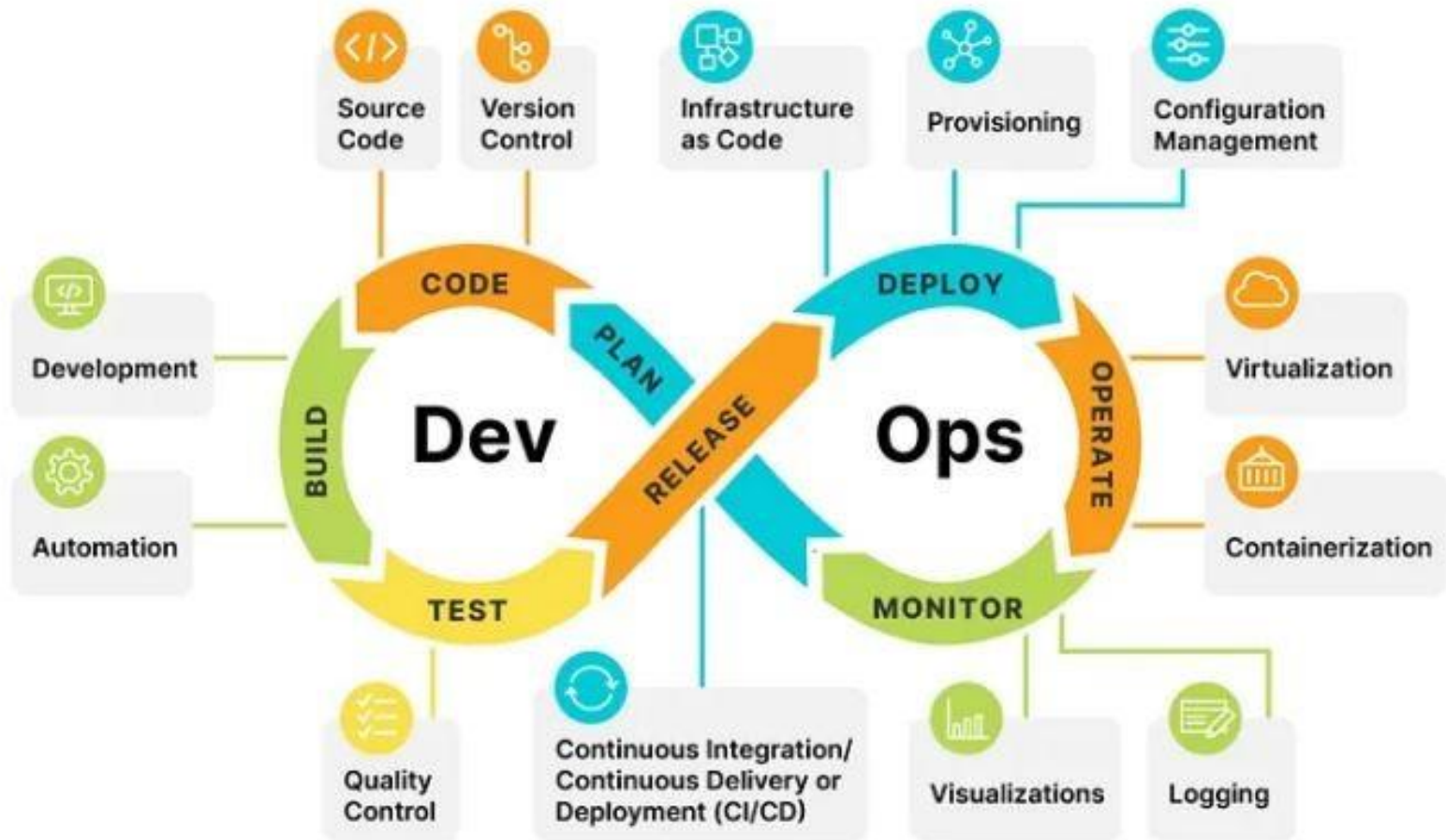
Dev vs Ops – Tư duy DevOps

Tư duy DevOps:



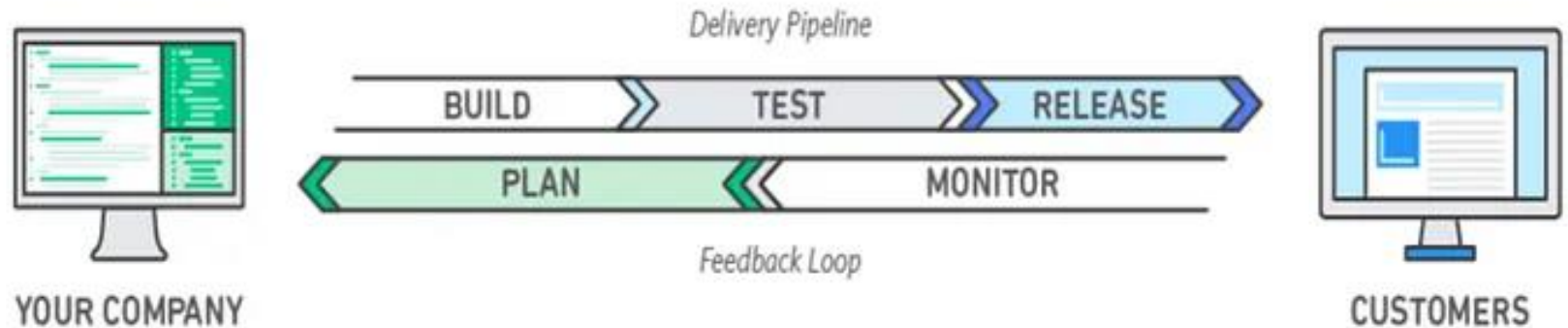
Dev vs Ops – Tư duy DevOps

Chu trình phát triển phần mềm với DevOps:



Dev vs Ops – Tư duy DevOps

Chu trình phát triển phần mềm với DevOps:



Dev vs Ops – Tư duy DevOps

Tóm lại DevOps là gì?

DevOps = Văn hóa + Quy trình + Tự động hóa

DevOps với Agile và LEAN

- **Agile:** phát triển đúng thứ cần (iterative, sprint, phản hồi sớm)
- **Lean:** tối ưu dòng chảy, giảm lãng phí (chờ đợi, chuyển giao, làm lại)
- **DevOps:** tự động hoá & vận hành để deploy liên tục (CI/CD, feedback từ production)
- **Kết hợp:** phát triển và giao đúng thứ cần với tốc độ nhanh và ổn định

DevOps với Security - DevSecOps

- **Truyền thống:** Security kiểm tra cuối → phát hiện trễ → sửa tổn kém, trễ release
- **DevSecOps:** security được tự động hoá và chạy từ sớm

DevOps với Security - DevSecOps

Pipeline:

Code → **Build** → **Test** → **Package** → **Deploy** → **Run**

Có thể đưa các “security checkpoints” vào:

- dưới **Code**: SAST + Secret scan (ngăn lộ API key/token lên Git)
- dưới **Build/Package**: Dependency scan + Image scan
- dưới **Deploy**: IaC/Policy check
- dưới **Run**: Monitoring/Alert (security events)